

TECH

JAVASCRIPT FUNDAMENTALS

EVENT LOOP / ASYNC

CLOSURES / SCOPE / MEMORY

PROTOTYPES / OBJECTS / CLASSES

CLOSURES / SCOPE / MEMORY

20 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК learn.javascript.ru

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 ЛЁГКИЙ	БАЗА CLOSURE r / let / const Q1 · Q6 · Q7	LEXICAL SCOPE	va	Фундамент темы: что такое замыкание, как ищется переменная, чем отличаются три ключевых слова. Без этого дальше му- тно.
ДЕНЬ 2 СРЕДНИЙ	STALE CLOSURE Q2 · Q3	БАГИ ЗАМЫКАНИЙ		Где замыкания ломают логику и что такое stale closure. Самая частая категория ошибок при работе с замыканиями.
ДЕНЬ 3 ТЯЖЁЛЫЙ	STALE CLOSURE В REACT HOOKS ЧИКИ СОБЫТИЙ Q4 · Q5	ОБРАБОТ		Коронный практический вопрос – почему useState даёт старое значение в setTimeout/обработчике, и как это лечить.
ДЕНЬ 4 ТЯЖЁЛЫЙ	ПАМЯТЬ GC Q9 · Q10 · Q11	MEMORY LEAK		Как замыкания держат память, какие источники утечек, что мешает сборщику мусора освободить объекты.
ДЕНЬ 5 СРЕДНИЙ	TDZ СРАВНЕНИЕ ОБЪЕКТОВ AL EQUALITY Q8 · Q15 · Q16	REFERENTI		Temporal dead zone и нюансы сравнения – почему { } !== { } и что такое ссылочное равенство.
ДЕНЬ 6 СРЕДНИЙ	КОПИРОВАНИЕ cturedClone Q12 · Q13 · Q14	SHALLOW vs DEEP	stru	Shallow vs deep, structured cloning, и почему JSON.parse(JSON.stringify) – не панацея.
ДЕНЬ 7 ЛЁГКИЙ	ИММУТАБЕЛЬНОСТЬ Map Q17 · Q18 · Q19 · Q20	РЕАКТ	Map / Weak	Закрепление: почему мутация ломает React-рендер, зачем immutability, чем Map отличается от объекта, когда WeakMap.

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ

ДЕНЬ 7 – БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1

НАГРУЗКА

ЛЁГКИЙ

3 ВОПРОСА

БАЗА**CLOSURE****LEXICAL SCOPE****var / let / const****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Фундамент темы: что такое замыкание, как ищется переменная, чем отличаются три ключевых слова. Без этого дальше мутно.

ВОПРОСЫ ДНЯ

Q01 Что такое closure?

Q06 Как работают lexical scope и scope chain?

Q07 Чем var, let, const отличаются с точки зрения scope?

Что такое closure?

ОПИСАНИЕ

Closure (замыкание) — это функция вместе с её лексическим окружением. То есть функция, которая помнит переменные из того места, где она была объявлена, даже если её вызывают далеко оттуда и через долгое время. В JavaScript замыкания — это не отдельная фишка, а следствие того, как устроена область видимости: каждая функция при создании запоминает ссылку на лексическое окружение, в котором она была определена. Эта ссылка живёт, пока существует сама функция.

КАК РАБОТАЕТ

- 01 Когда движок исполняет код функции, он создаёт для неё лексическое окружение (Lexical Environment) — объект с переменными и ссылкой на родительское окружение.
- 02 Внутренние функции получают невидимое поле `[[Environment]]` — ссылку на лексическое окружение, в котором их объявили.
- 03 При вызове внутренняя функция создаёт собственное окружение, родителем которого становится тот самый сохранённый `[[Environment]]` — отсюда доступ к переменным внешней функции.
- 04 Если внешняя функция уже завершилась, но внутренняя ещё жива (например, её сохранили в переменной или вернули наружу) — её лексическое окружение НЕ удаляется, потому что на него есть ссылка из внутренней функции.
- 05 Замыкание создаётся всегда — даже если переменных снаружи не используется. Просто движок умеет оптимизировать неиспользуемые.
- 06 Каждый вызов внешней функции создаёт НОВОЕ лексическое окружение и, соответственно, НОВОЕ замыкание для возвращаемой внутренней функции.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Счётчики и фабрики: `makeCounter()`, `makeAdder(n)` — генерируют функцию, помнящую своё начальное состояние.
- 02 Приватные данные: переменные внутри замыкания недоступны снаружи, доступ только через возвращённые методы.
- 03 Каррирование и частичное применение: `f(a)(b)(c)` хранит промежуточные аргументы в замыкании.
- 04 Колбэки в `setTimeout`, `addEventListener`, `.then`-цепочках — каждый из них замыкается над переменными своего места создания.
- 05 Любой React-хук — `useState`, `useEffect`, `useMemo` — это работа замыканий внутри функционального компонента.

ПОДВОДНЫЕ КАМНИ

- 01 Замыкание держит ВСЁ лексическое окружение, не только явно использованные переменные. Это может неожиданно удерживать память (см. Q9).

- 02 Каждый вызов внешней функции создаёт новое замыкание – если из 1000 кликов вы возвращаете 1000 функций, у вас 1000 разных копий окружения.
- 03 Внутри одного замыкания изменение переменной видно во всех ссылающихся на него функциях – это и фишка, и источник багов.
- 04 В цикле for с var = одно замыкание на всю итерацию (см. Q7), что обычно не то, чего хочешь.

КОД

// Классический пример – счётчик

```
function makeCounter() {
  let count = 0;
  return function () {
    return ++count;
  };
}
const c = makeCounter();
c(); // 1
c(); // 2
// `count` живёт в замыкании, недоступна снаружи
```

// Каждый вызов – новое замыкание

```
const c1 = makeCounter();
const c2 = makeCounter();
c1(); // 1
c1(); // 2
c2(); // 1 ← независимое окружение
```

// Замыкание над аргументом

```
function makeAdder(n) {
  return function (x) {
    return x + n;
  };
}
const add5 = makeAdder(5);
add5(3); // 8
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем замыкание в JS отличается от замыканий в Python / Lua?
- 02 Что такое [[Environment]] и зачем оно?
- 03 Можно ли через замыкание реализовать «приватные поля» класса?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/closure>

Как работают lexical scope и scope chain?

ОПИСАНИЕ

Lexical scope (лексическая область видимости) означает, что видимость переменной определяется местом её ОБЪЯВЛЕНИЯ в тексте программы, а не местом ВЫЗОВА функции. Scope chain (цепочка областей видимости) — это последовательность вложенных лексических окружений, по которой движок ищет переменную: сначала в текущем, потом в родительском, потом дальше до глобального. Не нашёл — ReferenceError. JS использует лексическую область видимости, в отличие от динамической, которая встречается в некоторых других языках.

КАК РАБОТАЕТ

- 01 При создании функции запоминается её внешнее окружение — то, в котором она объявлена. Это и есть lexical scope.
- 02 При вызове создаётся новое окружение для самой функции (параметры, var/let/const, this, arguments), родителем становится сохранённое внешнее окружение.
- 03 Поиск переменной: текущее окружение → родитель → дед → глобальное → не нашёл = ReferenceError. Это и есть scope chain.
- 04 В строгом режиме чтение необъявленной переменной всегда ReferenceError. Без strict — иногда происходит автосоздание глобальной (легаси-поведение).
- 05 Глобальное окружение в браузере — window/globalThis, в Node — globalThis (в модулях — есть нюансы, var на верхнем уровне модуля НЕ становится свойством globalThis).
- 06 Динамическая область видимости (как в bash или Lisp) — когда функция «видит» переменные ВЫЗЫВАЮЩЕГО кода. В JS такого нет, и слава богу.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Понимание scope chain нужно при работе с замыканиями — это буквально та цепочка, которая делает замыкания возможными.
- 02 При дебаге «откуда взялось это значение» — вопрос почти всегда сводится к scope chain.
- 03 При проектировании модулей: чем меньше глобальных переменных, тем проще цепочка, тем меньше неожиданностей.
- 04 Бандлеры (webpack, vite) оборачивают модули в IIFE — функции-обёртки, чтобы создать изолированный scope для переменных модуля.

ПОДВОДНЫЕ КАМНИ

- 01 Hoisting (поднятие) — var-переменные «видны» с начала функции, но до фактической строки объявления равны undefined. Часто путает новичков.
- 02 let/const тоже поднимаются, но с TDZ (см. Q8) — обращение до объявления = ReferenceError.

- 03 Функции-объявления (function f(){})) поднимаются целиком – их можно вызвать выше места объявления.
- 04 Глобальное замусоривание – переменная, объявленная без var/let/const внутри функции (в нестрогом), уйдёт в глобальный scope. Strict mode это запрещает.
- 05 Стрелочные функции не имеют собственного this/arguments – они берут эти значения из лексического scope. Иногда это спасение, иногда сюрприз.

КОД

// Lexical scope – место объявления, не вызова

```
let x = 'outer';
function getX() { return x; }
function caller() {
  let x = 'inner';
  return getX();
}
caller(); // 'outer'
// getX видит x по месту объявления, а не вызова
```

// Scope chain – поиск вверх

```
const a = 1;
function outer() {
  const b = 2;
  function inner() {
    const c = 3;
    return a + b + c; // a из глобального, b из outer
  }
  return inner();
}
```

// Скрытие имени (shadowing)

```
const x = 1;
function f() {
  const x = 2; // перекрывает внешний x
  return x;    // 2
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем лексическая область видимости отличается от динамической?
- 02 Что такое hoisting и для каких объявлений он работает?
- 03 Что вернёт globalThis в Node-модуле и в скрипте?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/closure>

Чем `var`, `let`, `const` отличаются с точки зрения `scope`?

ОПИСАНИЕ

`var` – старое объявление переменной с `function scope`: видна на протяжении всей функции, в которой объявлена, независимо от блоков. `let` и `const` – блочные (`block scope`), видны только внутри `{ ... }` и циклов. `const` запрещает переприсваивание, но НЕ запрещает мутацию объекта/массива. Дополнительные отличия: `var` `hoist`-ится с инициализацией `undefined`, `let/const` – с TDZ (см. Q8); `var` на верхнем уровне скрипта пишется в `window/globalThis`, `let/const` – нет.

КАК РАБОТАЕТ

- 01 `var`: `function scope`. Объявление поднимается к началу функции, значение доступно сразу как `undefined`.
- 02 `let`: `block scope` (`{ ... }` / циклы / `if`). Поднимается, но в TDZ до строки объявления – обращение даёт `ReferenceError`.
- 03 `const`: то же что `let` по `scope`, плюс запрет переприсваивания (не `reassign`). НЕ замораживает значение – для `immutable` объекта нужен `Object.freeze`.
- 04 В цикле `for-let` на каждой итерации создаётся НОВАЯ переменная, за которой замыкаются `callback`'и внутри тела. Для `var` – одна общая переменная, поэтому все колбэки видят финальное значение.
- 05 На верхнем уровне скрипта `var f` становится `window.f`, `let/const` – нет (они пишутся в `Module/Script Record`, а не в `Object Environment`).
- 06 В классе и модуле всегда строгий режим по умолчанию: чтение необъявленной переменной – `ReferenceError`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 В современном коде использовать `let` и `const`, `var` – только в легаси или в специфичных хаках с `function-scope`.
- 02 По умолчанию писать `const`, ослаблять до `let`, если нужна переприсваиваемость.
- 03 `var` до сих пор встречается в старом коде, в IIFE-паттернах, в полифиллах и в TS-выводе для ES5.

ПОДВОДНЫЕ КАМНИ

- 01 Классическая ловушка: `for (var i = 0; i < 3; i++) setTimeout(() => log(i))` – выведет три раза 3, потому что `var` – общая. С `let` – выведет 0, 1, 2.
- 02 `const protected obj` – путаются: `const` не означает `immutable`. `obj.x = 1` – можно. `const obj = ...` – нельзя.
- 03 `Hoisting var`-переменных позволяет читать их до объявления (будет `undefined`), что часто маскирует баги.
- 04 `let` и `const` в строгих режимах – обязательны; код, ассоциирующий через ``globalThis.x = 1``, не покрывает их.

05 TS-компилятор по умолчанию транпилирует `let` в `var` для старых таргетов – иногда возвращает к `function-scope`-семантике, если не следить.

КОД

// for + var – общая переменная

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}  
// 3, 3, 3
```

// for + let – новая на каждую итерацию

```
for (let i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}  
// 0, 1, 2
```

// const – не значит immutable

```
const obj = { x: 1 };  
obj.x = 2;      // ок  
obj = { x: 3 }; // TypeError: Assignment to constant
```

// Hoisting var до объявления

```
console.log(a); // undefined (a не ReferenceError)  
var a = 1;
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое TDZ и почему он есть у `let/const`, но не у `var`?
- 02 Можно ли `redeclare` `let/const` в одном `scope`? Что выдаст?
- 03 Куда уходит `var` на верхнем уровне модуля и скрипта?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/var>

STALE CLOSURE**БАГИ ЗАМЫКАНИЙ****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Где замыкания ломают логику и что такое stale closure. Самая частая категория ошибок при работе с замыканиями.

ВОПРОСЫ ДНЯ

Q02 Где closures приводят к багам?

Q03 Что такое stale closure?

ВОПРОС Q02

Где closures приводят к багам?

ОПИСАНИЕ

Замыкания сами по себе не багуют — баги вылезают из-за неаккуратной работы с переменными, которые они захватывают. Самые частые источники: общая переменная в цикле через `var`, захват «свежей» переменной, которая на самом деле уже поменялась, удержание объектов в памяти через `case-функции` (`memory leak`), и `stale closure` в `React-хуках`. Понимание этих паттернов помогает не наступать на грабли и быстро дебажить чужой странный код.

КАК РАБОТАЕТ

- 01 Замыкание захватывает не значение переменной, а саму переменную (её слот в окружении). Поэтому если переменная потом меняется, замыкание видит новое значение.
- 02 В `for + var` все колбэки шарят одну переменную `i` — отсюда классическая ошибка с `setTimeout` в цикле.
- 03 В `for + let` на каждой итерации создаётся новая переменная, и проблема пропадает.
- 04 Замыкание над переменной с большим объектом удерживает этот объект в памяти, пока живо хоть одно замыкание — источник утечек (см. Q9).
- 05 В `React` функциональный компонент = функция, при каждом рендере создаются новые переменные `state` и `props`, и хуки замыкаются над ними. Старая копия колбэка может остаться и видеть старые значения — отсюда `stale closure`.
- 06 Колбэки, переданные в `setInterval/addEventListener` один раз и не обновляемые, замыкаются над переменными того момента регистрации — со временем эти данные устаревают.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Понимание помогает дебажить «почему я вижу старое значение?» — обычно это `stale closure`.
- 02 Помогает выбирать правильный инструмент: `useCallback` с правильным `deps`-массивом, `useRef` для мутабельной свежей ссылки.
- 03 При обработке потоков событий и таймеров — сразу проверять, не замыкается ли колбэк над устаревшей переменной.

ПОДВОДНЫЕ КАМНИ

- 01 `for + var` в больших циклах — базовая ошибка JS. Лечится переходом на `let`.
- 02 Объект, удерживаемый замыканием, не освобождается. Если замыкание попало в долгоживущий обработчик — память течёт.
- 03 В `React`: `useEffect` без зависимостей, который читает `state` — увидит самое первое значение и больше никогда не обновится.
- 04 Внутри замыкания изменения переменной видны всем — это и фича, и источник «магических» багов, когда из одного места меняется состояние, на которое полагался

другой замыкан.

05 React-эффект, который замкнулся над state, иногда лечится не добавлением state в deps, а функциональным сеттером setState(prev => ...).

код

// for + var – классическая ошибка

```
const buttons = [...];
for (var i = 0; i < buttons.length; i++) {
  buttons[i].onclick = () => alert(i);
}
// все кнопки покажут buttons.length, не свой индекс
```

// Замыкание держит лишний объект

```
function makeHandler(bigData) {
  return () => console.log('clicked');
}
btn.onclick = makeHandler(hugeArray);
// hugeArray НЕ освободится, пока живёт onclick,
// даже если внутри handler он не используется
```

// React stale-closure (мини-пример)

```
function Counter() {
  const [n, setN] = useState(0);
  useEffect(() => {
    const id = setInterval(() => console.log(n), 1000);
    return () => clearInterval(id);
  }, []); // n не в deps
  // setInterval всегда печатает 0
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Как отловить утечку памяти, идущую через замыкание?
- 02 Почему стрелочные функции в обработчиках событий часто помогают?
- 03 Что вернёт console.log после for + var setTimeout – и почему?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/closure>

ВОПРОС Q03

Что такое stale closure?

ОПИСАНИЕ

Stale closure (устаревшее замыкание) — это ситуация, когда функция продолжает использовать значения из своего лексического окружения, но эти значения уже не актуальны. То есть функция «застряла» в прошлом моменте и не видит обновлённого состояния. Чаще всего возникает когда замыкание создано один раз и переиспользуется (обработчик клика, setInterval-колбэк, useEffect без правильных зависимостей), а данные вокруг продолжают меняться. Лечится либо обновлением замыкания (пересоздать колбэк при изменении), либо использованием мутабельной ссылки (ref), либо функциональным апдейтом (`prev => ...`).

КАК РАБОТАЕТ

- 01 Замыкание захватывает переменную, не значение. Но если переменная — это локальная константа из конкретного рендера, она там и застряла.
- 02 В React функциональном компоненте каждый рендер создаёт НОВЫЕ переменные state и props. Старый колбэк всё ещё ссылается на переменные старого рендера.
- 03 В useEffect без зависимости на state — эффект создаётся один раз с текущим значением state, дальше не пересоздаётся, и колбэк навсегда видит первое значение.
- 04 В addEventListener тот же риск — функция-колбэк зарегистрирована один раз, замыкается на переменные того момента и не обновляется.
- 05 Лечения три: 1) добавить переменную в deps, чтобы колбэк пересоздавался; 2) использовать ref (useRef) — мутабельный контейнер, ссылка на который не меняется, а .current — да; 3) использовать функциональный апдейт `setState(prev => ...)` — prev всегда свежее.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 useEffect, useCallback, useМемо — каждый раз думать, правильные ли deps. Линтер eslint-plugin-react-hooks помогает.
- 02 setInterval и setTimeout, регистрируемые один раз и читающие состояние — кандидаты на stale closure.
- 03 Обработчики событий, повешенные через addEventListener вне хуков — нужно перерегистрировать при смене зависимости.

ПОДВОДНЫЕ КАМНИ

- 01 Линтер react-hooks/exhaustive-deps часто советует добавить зависимость, и это правильный совет в большинстве случаев. Игнорировать его без понимания — риск.
- 02 Добавление state в deps делает useEffect перезапускающимся на каждом изменении state — иногда это не то, что нужно. Альтернатива — useRef.
- 03 useRef для свежего значения — рабочий приём, но усложняет код: ref не вызывает ререндер, и иногда нужен оба — state для рендера и ref для свежей ссылки.

04 `setState(prev => ...)` – самый чистый способ, когда новое значение зависит от старого. Не нужно даже добавлять `state` в `deps`.

05 В классических компонентах (классы) `stale closure` тоже встречается, но реже – там обычно есть `this.state`, который всегда свежий.

КОД

// Базовый пример `stale closure` в `setInterval`

```
function App() {
  const [n, setN] = useState(0);
  useEffect(() => {
    const id = setInterval(() => {
      console.log(n); // всегда 0
    }, 1000);
    return () => clearInterval(id);
  }, []);
  return <button onClick={() => setN(n + 1)}>+</button>;
}
```

// Лечение через зависимость

```
useEffect(() => {
  const id = setInterval(() => {
    console.log(n);
  }, 1000);
  return () => clearInterval(id);
}, [n]); // эффект пересоздается при смене n
```

// Лечение через `ref`

```
const nRef = useRef(0);
useEffect(() => { nRef.current = n; }, [n]);
useEffect(() => {
  const id = setInterval(() => {
    console.log(nRef.current); // всегда свежее
  }, 1000);
  return () => clearInterval(id);
}, []);
```

// Лечение функциональным апдейтом

```
setN(prev => prev + 1); // не зависит от замыкания
```

МОГУТ ДОСПРОСИТЬ

01 Когда выбрать `ref`, а когда – добавление в `deps`?

02 Почему функциональный апдейт `setState` – вообще не страдает `stale`?

03 Может ли `stale closure` случиться без `React`?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/closure>

ДЕНЬ 3

НАГРУЗКА

ТЯЖЁЛЫЙ

2 ВОПРОСА

STALE CLOSURE В REACT HOOKS

ОБРАБОТЧИКИ СОБЫТИЙ

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Коронный практический вопрос — почему `useState` даёт старое значение в `setTimeout`/обработчике, и как это лечить.

ВОПРОСЫ ДНЯ

Q04 Почему stale closure часто возникает в React hooks?

Q05 Как избежать stale closure в обработчиках событий?

Почему stale closure часто возникает в React hooks?

ОПИСАНИЕ

Главная причина — модель функционального компонента в React. При каждом рендере React заново вызывает функцию компонента, и внутри создаются новые переменные для state, props и любых локальных вычислений. Каждый колбэк, объявленный в этой функции, замыкается над переменными именно этого рендера. Если такой колбэк сохраняется и переиспользуется (через useEffect без deps, через event listener, в setTimeout), он навсегда остаётся с теми переменными, не зная о том, что компонент перерендерился. Это и есть stale closure в React-форме.

КАК РАБОТАЕТ

- 01 React функциональный компонент = функция, и каждый рендер — это её новый вызов. Переменные внутри — это переменные данного рендера.
- 02 Хуки сохраняют свои данные ВНЕ функции (в фибере), но ВЫДАЮТ их как локальные переменные внутри — поэтому useState(...) каждый раз даёт пару из текущей версии state.
- 03 useEffect берёт колбэк-функцию по ссылке. Если deps не поменялись — эффект не пересоздаётся, и старый колбэк продолжает работать с переменными старого рендера.
- 04 useCallback — это просто способ запомнить ту же ссылку колбэка между рендерами, но захваченные переменные всё равно от того рендера, для которого deps были актуальны.
- 05 useRef возвращает один и тот же объект на всех рендерах. Записав в .current новое значение, можно «достать его из будущего» в старом замыкании.
- 06 Главный инструмент дебага stale closure в React — линтер eslint-plugin-react-hooks/exhaustive-deps.
- 07 В Strict Mode React 18 эффекты в dev запускаются дважды — это специально, чтобы ловить stale-closure-баги без cleanup.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой useEffect с таймером, обработчиком события, подпиской на стрим — кандидат на stale.
- 02 useCallback / useMemo — правильно подобрать deps, чтобы колбэк/значение пересчитывались.
- 03 Кастомные хуки часто инкапсулируют решение: useEvent, useLatest, useEventCallback — все они под капотом это та же история с ref + эффектом.
- 04 В react-query / SWR stale-replay не страшен потому что данные читаются из кэша по ключу — без замыканий над state.

ПОДВОДНЫЕ КАМНИ

- 01 Не добавлять переменную в deps только потому, что эффект перезапускается — это маскировка проблемы. Лучше переписать на функциональный апдейт или ref.

02 `useCallback` без правильных `deps` создаёт ту же проблему, что и обычная функция: захватывает старые переменные.

03 `ref` — мощный инструмент, но злоупотребление превращает функциональный компонент в имитацию класса. Использовать только когда правда нужно.

04 В Strict Mode React 18 эффекты двойные — иногда это путает, потому что баг видно только в dev.

05 Серверные компоненты (RSC) не имеют хуков и `stale closure`, но клиентские внутри них — имеют.

код

// Пустые `deps` + чтение `state` — типичная ошибка

```
function Chat() {
  const [msg, setMsg] = useState('');
  useEffect(() => {
    socket.on('message', () => {
      console.log(msg); // всегда ''
    });
  }, []);
}
```

// Решение через `useRef` + `sync`

```
function Chat() {
  const [msg, setMsg] = useState('');
  const msgRef = useRef(msg);
  useEffect(() => { msgRef.current = msg; }, [msg]);
  useEffect(() => {
    socket.on('message', () => {
      console.log(msgRef.current); // свежее
    });
  }, []);
}
```

// Кастомный `useLatest`-хук

```
function useLatest(value) {
  const ref = useRef(value);
  ref.current = value;
  return ref;
}
// внутри компонента:
const latestMsg = useLatest(msg);
// использовать latestMsg.current внутри long-living коллбэков
```

// Функциональный апдейт без замыкания

```
// не зависит от текущего n
setN(prev => prev + 1);
```

МОГУТ ДОСПРОСИТЬ

01 Что такое `useEvent` / `useEventCallback` и какую проблему он решает?

02 Почему в Strict Mode React 18 эффекты dev-запускаются дважды?

03 Чем `useCallback` отличается от `useMemo` в контексте `stale closure`?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/closure>

ВОПРОС Q05

Как избежать stale closure в обработчиках событий?

ОПИСАНИЕ

В обработчиках событий (`addEventListener`, `onClick` и подобных) stale closure возникает, когда колбэк зарегистрирован один раз и со временем читает уже устаревшие данные. Лечится либо перерегистрацией обработчика при изменении зависимости, либо использованием `ref`-контейнера, который содержит свежую ссылку на нужное значение, либо чтением свежего state через функциональный апдейт. В React, помимо этого, есть паттерн `useEvent / useEventCallback`: специальный хук, который возвращает стабильную функцию, всегда вызывающую свою последнюю версию.

КАК РАБОТАЕТ

- 01 `addEventListener` регистрирует ссылку на функцию. Если функция замкнулась над переменной первого рендера, она навсегда видит старое значение.
- 02 Снять обработчик можно только тем же объектом-функцией, что был передан в `addEventListener`. Поэтому пересоздание колбэка обычно требует `removeEventListener-then-add`.
- 03 `useRef`-подход: храним свежее значение в `ref.current`, в обработчике читаем `ref.current`. Обработчик регистрируется один раз, но всегда видит актуальное.
- 04 Функциональный апдейт `setState(prev => ...)` не нужен `deps` на state – `prev` приходит из React, а не из замыкания.
- 05 `useEvent` (RFC, реализован в виде `userland`-хуков): возвращает стабильную функцию, под капотом – `useRef` + эффект, который обновляет `ref` на каждый рендер.
- 06 В чистом DOM (без React): пересоздать обработчик и заменить регистрацию, либо хранить актуальное в общем объекте-замкнутом.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Кнопка Submit, читающая текущий ввод формы – обычно через `controlled state` и обработчик. Stale ловится когда state захватывается в `onSubmit`.
- 02 Подписка на `keydown/scroll/wheel`: один раз вешаем, нужно, чтобы он видел свежий state.
- 03 Сетевой колбэк, читающий `props/state`: при долгой работе `props` могут смениться.
- 04 Слушатель Web Socket-сообщений и push-уведомлений.

ПОДВОДНЫЕ КАМНИ

- 01 Перерегистрация на каждый рендер – это потеря производительности и потенциальные проблемы с порядком обработки.
- 02 `Ref+sync` через эффект делается отдельным `useEffect`: `useEffect(() => { ref.current = value; }, [value])`.
- 03 `useEvent` в Production-React официально не введён (на момент написания), но есть готовые `userland` реализации (`use-event`, `@radix-ui/react-use-callback-ref`).

04 Без React: в IIFE-замкнутом модуле часто хранят `latestState` и читают его внутри обработчика. Это работает, но напоминает костыль.

05 Стрелочные функции в JSX (`onClick={() => ...}`) пересоздаются на каждом рендере — но это и есть «всегда свежий колбэк», `stale` тут не возникает. Минус — мемоизация ниже по дереву ломается.

КОД

// Чистый DOM — ref-контейнер для свежего значения

```
let latestUser = currentUser;
function update(u) { latestUser = u; }
btn.addEventListener('click', () => {
  send(latestUser);
});
```

// React: `useEvent`-подобный хук

```
function useEventCallback(fn) {
  const ref = useRef(fn);
  useEffect(() => { ref.current = fn; });
  return useCallback((...a) => ref.current(...a), []);
}
// onClick={onSendStable}, всегда свежая логика
```

// Функциональный апдейт

```
// не нужен ref/deps:
<button onClick={() => setN(prev => prev + 1)}>+</button>
```

// Перерегистрация — простой и часто достаточный приём

```
useEffect(() => {
  function handler() { console.log(n); }
  window.addEventListener('keydown', handler);
  return () => window.removeEventListener('keydown', handler);
}, [n]);
```

МОГУТ ДОСПРОСИТЬ

01 Когда выбрать `useEvent`, а когда — простую перерегистрацию?

02 Почему стрелочная функция в JSX каждый рендер не страшна?

03 Что произойдёт, если забыть `removeEventListener` в `cleanup`?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/closure>

ДЕНЬ 4 НАГРУЗКА ТЯЖЁЛЫЙ 3 ВОПРОСА

ПАМЯТЬ GC MEMORY LEAK

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Как замыкания держат память, какие источники утечек, что мешает сборщику мусора освободить объекты.

ВОПРОСЫ ДНЯ

- Q09 Как closures могут удерживать память?
- Q10 Как найти memory leak в JavaScript?
- Q11 Что может помешать garbage collection?

Как closures могут удерживать память?

ОПИСАНИЕ

Замыкание держит ссылку на лексическое окружение, в котором его создали. Это значит — пока живёт сама функция-замыкание, живёт и всё её окружение, со всеми объектами, на которые в этом окружении есть ссылки. Если такая функция попадёт в долгоживущее место (обработчик события, поле объекта, глобальная переменная), все объекты, которые она «помнит», тоже остаются в памяти и не освобождаются сборщиком мусора. Так через замыкания получаются типичные утечки памяти в SPA — компонент удалён из DOM, а данные за ним всё ещё висят, потому что подписка осталась.

КАК РАБОТАЕТ

- 01 Garbage collector (GC) удаляет объекты, на которые больше никто не ссылается. JS использует mark-and-sweep: достижимое из root-set остаётся, недостижимое — удаляется.
- 02 Замыкание содержит в `[[Environment]]` ссылку на лексическое окружение. Окружение содержит ссылки на переменные. Переменные — на объекты.
- 03 Если функция-замыкание достижима — достижимы и все её захваченные переменные. Поэтому большой объект может «жить» из-за одного маленького колбэка.
- 04 Современные движки оптимизируют: если конкретная переменная из замыкания НЕ используется внутри, GC может её освободить. Но это эвристика — на неё нельзя полагаться.
- 05 Часто проблема не в самом замыкании, а в том, что кто-то хранит ссылку на это замыкание (например, `addEventListener` не сняли, или пушит в массив-кэш).
- 06 `WeakMap/WeakRef` позволяют ссылаться на объект «слабо» — без удержания. GC может убрать объект даже если есть слабая ссылка.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Анализ утечек памяти в Chrome DevTools / Memory tab — Heap snapshot, Allocation timeline.
- 02 Long-living подписки (`WebSocket`, `setInterval`, `EventTarget`) — проверять, что `cleanup` снимает их.
- 03 Кэши и мемоизация — следить, чтобы не накапливалось бесконечно. `WeakMap` часто решает.
- 04 Обработчики DOM-событий внутри SPA-компонентов — их обязательно снимать при `unmount`.

ПОДВОДНЫЕ КАМНИ

- 01 Линкуете обработчик и сразу теряете ссылку — снять не получится. Всегда сохраняйте функцию в переменную.
- 02 Замыкание в `setInterval` живёт до `clearInterval`. Если ссылку на интервал потерять — он будет работать вечно.

03 Циклические ссылки больше не проблема для современного GC (mark-and-sweep их обрабатывает). Кроме редких case с DOM <-> JS.

04 console.log замыкания в DevTools удерживает его – иногда это сбивает анализ утечек. Закройте DevTools перед измерением или используйте отдельный Heap snapshot.

05 Каскад: компонент удалён, но в нём была подписка с большим колбэком, который замыкается на пропсы → весь поддерево props сохраняется в памяти, пока подписка не снята.

КОД

// Утечка через долгоживущий обработчик

```
function attach(socket, page) {
  const heavy = page.heavyData; // 50MB
  socket.on('msg', () => {
    console.log('msg received');
    // heavy не используется, но захвачено в замыкании
  });
}
// page.heavyData не освободится, пока висит socket-обработчик
```

// Решение – обнулить ссылку явно

```
function attach(socket, page) {
  let heavy = page.heavyData;
  socket.on('msg', () => {
    console.log('msg received');
  });
  // если heavy внутри не нужно – можно обнулить
  heavy = null;
}
```

// Кэш через WeakMap – не удерживает ключи

```
const cache = new WeakMap();
function getMeta(obj) {
  if (!cache.has(obj)) cache.set(obj, calc(obj));
  return cache.get(obj);
}
// если obj больше нигде не нужен – GC его удалит,
// и запись в cache тоже исчезнет
```

МОГУТ ДОСПРОСИТЬ

01 Как найти такую утечку в DevTools?

02 Почему console.log замыкания в DevTools мешает анализу?

03 Чем WeakMap отличается от Map по поведению с GC?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/closure>

Как найти memory leak в JavaScript?

ОПИСАНИЕ

Memory leak в JS — это объекты, которые остаются в куче, хотя по бизнес-логике уже не нужны. Браузер сам не понимает, что они «лишние»: формально на них где-то есть ссылка, GC их не удалит. Поиск утечек обычно сводится к сравнению снимков кучи: сделали действие, очистили — снимок; повторили N раз — снимок; ищем объекты, число которых растёт. Дальше Chrome DevTools показывает retainer-цепочку — почему именно объект всё ещё достижим. Самые частые источники: подписки, замыкания в долгоживущих местах, кэши без ограничений, detached DOM-узлы, циклы между React-state и DOM.

КАК РАБОТАЕТ

- 01 Chrome DevTools → Memory tab → Heap snapshot. Сделать снимок, выполнить сценарий, очистить, ещё снимок. Сравнить — что осталось.
- 02 Allocation Timeline (Allocation instrumentation on timeline) — показывает аллокации на временной шкале с указанием стека вызовов.
- 03 Performance memory tab — графики heap size и DOM nodes по времени. Растёт линейно — есть утечка.
- 04 В Node — heap profiler через `--inspect`, либо `clinic.js`, либо `modules` вроде `memwatch-next`, `heapdump`.
- 05 `Performance.measureUserAgentSpecificMemory()` — браузерный API (требует `cross-origin isolation`), даёт точную цифру для вкладки.
- 06 В V8 GC можно вызвать вручную через `--expose-gc + global.gc()` (только в Node) или через DevTools кнопку «Collect garbage» — удобно для теста перед снимком.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Долгоживущие SPA, которые показывают значительное потребление памяти после долгого использования.
- 02 Регрессии после релизов — особенно если релиз менял обработчики событий или подписки.
- 03 Перед production-релизом — профилировать ключевые сценарии.
- 04 В Node-сервисах при росте RSS — `heapdump` + просмотр в Chrome DevTools (или `sentry-perf`, `datadog`).

ПОДВОДНЫЕ КАМНИ

- 01 DevTools удерживает в памяти открытые консольные ссылки на объекты — закройте Console / снимите `Pause-on-exception`.
- 02 Каждый снимок сам по себе ещё чуть подъедает память — это нормально, главное — устойчивая разница между ними.
- 03 Не каждое «неосвобождение» — leak. Иногда это просто ленивая инициализация. Проверять надо на повторяющихся циклах действий.

04 Detached DOM nodes – частый случай: узел удалили из DOM, но JS-ссылка на него сохранилась (например, в массиве). Видны в Heap snapshot как Detached HTMLElement.

05 Замыкания, идущие через цепочку bind/curry/wrap – могут быть трудно отслеживаемыми в retainer chain.

КОД

// Заметная утечка через массив-логгер

```
const log = [];  
function record(payload) {  
  log.push(payload); // НИКОГДА не очищается  
}  
// каждый record удерживает payload навсегда
```

// Detached DOM в JS-ссылке

```
const detached = [];  
function remove(el) {  
  detached.push(el); // удерживаем удалённый узел  
  el.remove();  
}  
// detached растёт, в Heap snapshot – Detached HTMLElement
```

// Profiling через --expose-gc в Node

```
// node --expose-gc app.js  
global.gc();  
console.log(process.memoryUsage().heapUsed);  
// провести сценарий  
global.gc();  
console.log(process.memoryUsage().heapUsed);  
// разница = утечка
```

МОГУТ ДОСПРОСИТЬ

01 Чем Heap snapshot отличается от Allocation timeline?

02 Что такое retained size и почему он важнее, чем shallow size?

03 Как поймать утечку в Node prod-сервисе без даунтайма?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/garbage-collection>

Что может помешать garbage collection?

ОПИСАНИЕ

GC удаляет объекты, на которые больше нет достижимых ссылок из root-set. Поэтому помешать ему может одно – оставшаяся достижимая ссылка, явная или скрытая. Самые частые «удержатели»: глобальные переменные, замыкания в долгоживущих местах, обработчики событий, таймеры/интервалы, подписки, кэши (особенно через Map), массивы-логи, ссылки на DOM в JS. WeakMap, WeakSet и WeakRef нужны как раз чтобы хранить «ненастойчивые» ссылки – GC может удалить объект, даже если на него есть слабая ссылка.

КАК РАБОТАЕТ

- 01 Корневое множество (roots): глобальные переменные (window, globalThis), стек активных функций, реестры WebAPI. Объект достижим, если до него есть путь от root.
- 02 Mark-and-sweep: GC проходит по графу объектов от root, помечает достижимые, остальные удаляет. Циклы (a → b, b → a) обрабатываются без проблем.
- 03 Generation GC (V8): новые объекты в Young Generation, чаще GC; долгоживущие переезжают в Old Generation, реже GC. Малые сборки быстрые, большие – дороже.
- 04 WeakMap/WeakSet: запись по ключу-объекту не удерживает сам ключ. Если на ключ нет других ссылок – запись пропадает.
- 05 WeakRef + FinalizationRegistry: ссылка, которая не мешает GC, плюс колбэк после сборки. Часто в кэшах.
- 06 FinalizationRegistry-колбэки гарантированно вызываются не сразу, и не всегда – на их жёсткое выполнение полагаться нельзя.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Аудит долгоживущих частей приложения: подписки, обработчики, глобальные кэши.
- 02 При проектировании кэша – выбор Map vs WeakMap зависит от того, нужно ли держать ключ.
- 03 При создании observer-паттерна – WeakRef помогает не удерживать наблюдателя.
- 04 В библиотеках (state-managers, реактивность) WeakMap часто используется для метаданных объекта без удержания.

ПОДВОДНЫЕ КАМНИ

- 01 Глобальная переменная, которой случайно присвоили большой объект – навсегда. Strict mode помогает не наплодить такие случайно.
- 02 Обработчик addEventListener не снимается автоматически. Снять можно только тем же объектом-функцией.
- 03 setInterval без clearInterval работает вечно и удерживает колбэк со всем, что тот замкнул.

04 console.log объекта в DevTools – DevTools удерживает его до закрытия. Логирование в проде – нет.

05 Detached DOM nodes – самый коварный случай. Узел удалили из DOM, но JS-ссылка осталась в массиве/state – нода в памяти.

06 Замыкание над переменной, которую внутри НЕ используем, в теории GC может выжать. Но это не гарантия – лучше обнулить ссылку явно.

КОД

// Map vs WeakMap – различие в поведении GC

```
const map = new Map();
const weak = new WeakMap();
{
  const obj = {};
  map.set(obj, 'A');
  weak.set(obj, 'B');
}
// после блока ссылка на obj пропала
// в weak запись исчезнет, в map – останется навсегда
```

// WeakRef для слабой ссылки

```
let target = { big: data };
const ref = new WeakRef(target);
target = null;
// ref.deref() может вернуть undefined,
// если GC уже сработал
```

// Корни – что точно удерживает

```
// глобально
globalThis.cache = bigData;
// в стеке активных функций
function hold() {
  const x = bigData;
  while (true) {} // bigData удерживается этой функцией
}
```

МОГУТ ДОСПРОСИТЬ

01 Чем WeakRef отличается от WeakMap по интерфейсу?

02 Что такое Generation GC в V8?

03 Какие гарантии (или их отсутствие) у FinalizationRegistry?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/garbage-collection>

TDZ

СРАВНЕНИЕ ОБЪЕКТОВ

REFERENTIAL EQUALITY

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Temporal dead zone и нюансы сравнения — почему `{ } !== { }` и что такое ссылочное равенство.

ВОПРОСЫ ДНЯ

Q08 Что такое temporal dead zone?

Q15 Как сравнивать объекты?

Q16 Что такое referential equality?

ВОПРОС Q08

Что такое temporal dead zone?

ОПИСАНИЕ

Temporal dead zone (TDZ) – это интервал в коде между началом блока и фактической строкой объявления переменной `let/const`, в течение которого переменная как бы «существует» (она уже поднята/*hoisted* в скоуп), но обращение к ней даёт `ReferenceError`. У `var` такого нет: до объявления она равна `undefined`, без ошибки. TDZ нужна, чтобы поймать опечатки и неправильный порядок объявлений на этапе исполнения, и чтобы `const`-семантика не выглядела размытой.

КАК РАБОТАЕТ

- 01 `let` и `const` поднимаются на верх своего блока (*block scope*). Но в отличие от `var`, они не инициализируются `undefined`.
- 02 Между началом блока и строкой объявления – TDZ. Любое чтение/запись = `ReferenceError: Cannot access 'x' before initialization`.
- 03 После строки объявления переменная инициализируется и становится доступна. Если объявлено без значения, оно `undefined` (для `let`) или `ReferenceError`-при-чтении (для `const`, потому что `const` обязан иметь значение при объявлении).
- 04 TDZ работает и для параметров функции в некоторых случаях: `function f(a = b, b = 1) {}` – в момент вычисления `a` переменная `b` ещё в TDZ.
- 05 В классе тоже есть подобие TDZ для полей и `super()`: обращение к `this` до `super()` – `ReferenceError`.
- 06 В `for-let` итерационная переменная тоже в TDZ для каждой новой итерации, но это редко чувствуется.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Помогает ловить опечатки имён на этапе исполнения.
- 02 Заставляет писать объявления выше их использования – лучше для читаемости.
- 03 В классах гарантирует, что `this` недоступен до `super()`.
- 04 В свежем коде – фактически всегда, потому что `let/const` – стандарт.

ПОДВОДНЫЕ КАМНИ

- 01 Сообщение об ошибке часто выглядит запутанно: «Cannot access 'x' before initialization». Это и есть TDZ.
- 02 *Hoisting* `let/const` есть, но не такой удобный как у `var`. Поэтому многие думают, что `let/const` «не поднимаются» – это упрощение, технически они поднимаются, но в TDZ.
- 03 Попытка вызвать класс до его объявления тоже даёт TDZ-ошибку: класс – это `let`-биндинг.
- 04 Стрелочная функция, объявленная через `const fn = () => ...`, не доступна до этой строки. `function fn() {}` – доступна сверху файла из-за *hoisting* функций.

05 Иногда TDZ маскирует другие проблемы: писали что-то по ленивой логике, а получили ошибку про переменную — а проблема выше по коду.

код

// Базовый TDZ

```
{
  console.log(x); // ReferenceError
  let x = 1;
}
```

// var — без TDZ

```
{
  console.log(y); // undefined
  var y = 1;
}
```

// const обязан иметь значение

```
{
  const z; // SyntaxError: missing = value
}
```

// Параметры функции — порядок важен

```
function f(a = b, b = 1) { return [a, b]; }
f(); // ReferenceError: b is not defined
```

// this до super() — TDZ

```
class A {}
class B extends A {
  constructor() {
    console.log(this); // ReferenceError
    super();
  }
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Поднимаются ли let и const? Как это совмещается с TDZ?
 - 02 Почему const x; — синтаксическая ошибка, а let x; — нет?
 - 03 Может ли TDZ возникать в стрелочных функциях?
-

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/var>

ВОПРОС Q15

Как сравнивать объекты?

ОПИСАНИЕ

В JS оператор `===` для объектов сравнивает только ссылки: два объекта равны только если это один и тот же объект в памяти. Поэтому `{ a: 1 } !== { a: 1 }`. Чтобы сравнить объекты по содержимому (структурное равенство), нужно написать или взять готовое: `lodash.isEqual`, `fast-deep-equal`, `JSON.stringify` (как ленивый и кривой вариант), либо ручной обход полей. Для каждого случая выбор разный – глубокое равенство объектов медленнее, чем сравнение ссылок, и его обычно избегают в горячем коде.

КАК РАБОТАЕТ

- 01 `===/==` над объектами – сравнение ссылок. Это поведение стандарта ECMAScript, без исключений.
- 02 `Object.is` – почти то же что `===`, плюс корректный `NaN === NaN (true)` и `-0 !== +0` (`Object.is` различает их).
- 03 Структурное равенство – обход всех ключей и сравнение значений. Рекурсивный, со всеми тонкостями: даты, мапы, сеты, регулярки.
- 04 `JSON.stringify`-сравнение – кривое: порядок ключей может не совпадать; функции/`Symbol`/`undefined` теряются; даты превращаются в строки. Только для очень простых данных.
- 05 `lodash.isEqual` / `fast-deep-equal` / `dequal` – производственные решения. Различаются по охвату типов и скорости.
- 06 Для больших структур стоит думать о хешировании или о `shape-IDs` (`immer + structural sharing`) – иначе сравнение становится узким местом.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Тесты – `assertEqual` / `toEqual` обычно делают глубокое сравнение под капотом.
- 02 Мемоизация – кэш по объекту-аргументу, проверка «не менялся ли вход».
- 03 `React.memo` / `shouldComponentUpdate` – обычно сравнивают `props` по ссылкам (`shallow`), а не глубоко, чтобы не терять перфоманс.
- 04 Diff алгоритмы (`Redux selectors`, `react-query`) – часто `shallow` вместо `deep` по тем же соображениям.

ПОДВОДНЫЕ КАМНИ

- 01 `JSON.stringify` теряет `undefined`, функции, `Symbol`, циклические ссылки (выбрасывает ошибку). Никогда не пишите `serialize`-сравнение для сложных данных.
- 02 `{ a: 1, b: 2 }` и `{ b: 2, a: 1 }` – `equal` по содержимому, но `JSON.stringify` даст разные строки. Нужен сортировщик ключей или нормальная либа.
- 03 Глубокое сравнение для огромных деревьев медленное. Часто проще сделать `immutable update` + ссылочное сравнение.

04 Сравнение Map/Set требует специального подхода – в lodash isEqual это есть, в JSON-варианте не работает вообще.

05 Регэкспы и даты сравниваются по ссылке через ===. lodash.isEqual умеет сравнить по value.

КОД

// === – это про ссылки

```
{ a: 1 } === { a: 1 } // false
const x = { a: 1 };
x === x;              // true
```

// Object.is – нюансы NaN и -0

```
NaN === NaN;          // false
Object.is(NaN, NaN);  // true
Object.is(0, -0);     // false
```

// Простое глубокое сравнение для plain-object

```
function deepEqual(a, b) {
  if (a === b) return true;
  if (typeof a !== 'object' || typeof b !== 'object'
      || a === null || b === null) return false;
  const ka = Object.keys(a), kb = Object.keys(b);
  if (ka.length !== kb.length) return false;
  return ka.every(k => deepEqual(a[k], b[k]));
}
```

// JSON-сравнение – ленивое и кривое

```
JSON.stringify(a) === JSON.stringify(b);
// порядок ключей, потеря типов, undefined, циклы...
```

МОГУТ ДОСПРОСИТЬ

01 Чем Object.is отличается от ===?

02 Какие крайние случаи плохо обрабатывает JSON.stringify-сравнение?

03 Почему React.memo делает shallow, а не deep?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/object-copy>

ВОПРОС Q16

Что такое `referential equality`?

ОПИСАНИЕ

`Referential equality` (ссылочное равенство) – равенство по ссылке, то есть «это один и тот же объект в памяти». Противоположность – `structural equality` (равенство по содержимому). В JS `===` для объектов делает именно ссылочное сравнение. Концепция важна для React и других реактивных систем: они часто решают, нужно ли перерисовать компонент, на основе ссылочного равенства `props/state`. Поэтому мутация массива/объекта без создания новой ссылки не вызывает обновление, а каждое расширение объекта через `spread` даёт новую ссылку – и обновление случается.

КАК РАБОТАЕТ

- 01 У объекта есть `identity` – его адрес в куче. Два литерала `{a:1}` и `{a:1}` имеют разные `identity` и не равны по ссылке.
- 02 `===`, `Object.is`, `Map.has`, `Set.has` работают по `identity` для объектов.
- 03 Прimitives (`string`, `number`, `boolean`, `bigint`, `symbol`) сравниваются по `value`, поэтому `'abc' === 'abc' = true`.
- 04 `React.memo` и `useMemo` полагаются на ссылочное равенство для пропуска рендера.
- 05 Чтобы сохранить ссылку при «обновлении» – нужно либо мутировать (но тогда `identity` та же, реактивность не работает), либо создать новый объект (тогда `identity` другая, реактивность работает).
- 06 `Immutable.js`, `Immer`, `structural sharing` – все они стараются сохранять `identity` тех частей дерева, которые не менялись, и менять `identity` у обновлённых веток.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 `React.memo` с дефолтным сравнением – `shallow` по `props`, то есть `identity` для каждого `props`.
- 02 `useMemo` / `useCallback` – мемоизация по ссылочному равенству `deps`.
- 03 `Redux selectors` с `reselect` – кэш по ссылке.
- 04 `Set/Map` с объектами в качестве ключей – по `identity`.

ПОДВОДНЫЕ КАМНИ

- 01 Inline-литералы в JSX (`<C data={{a:1}} />`) – каждый рендер новая `identity`, `React.memo` не работает. Решение – `useMemo`.
- 02 Аналогично inline-стрелочные функции – пересоздаются на каждый рендер. `useCallback` помогает.
- 03 `Spread` не делает `deep clone` – внутренние объекты остаются теми же. `Identity` у вложенных не меняется.
- 04 `Map` с объектом-ключом – после потери исходной ссылки на объект ключ потерян (не достать иначе). Поэтому `WeakMap`.

05 Сильно опираться на identity опасно при работе с serialization (JSON, structuredClone) – после round-trip identity всегда новая.

КОД

// Identity vs structural

```
{ } === { }; // false (разная identity)
const a = { };
a === a; // true
```

// Spread меняет identity верхнего уровня

```
const o = { x: 1, nested: { y: 2 } };
const n = { ...o };
n === o; // false
n.nested === o.nested; // true ← вложенный – тот же
```

// React.memo – по identity

```
const Item = React.memo(({ data }) => ...);
// если data – литерал {a:1} в JSX,
// React.memo НЕ работает – каждый рендер новая identity
```

// Set с объектами – identity-based

```
const s = new Set();
s.add({a: 1});
s.has({a: 1}); // false ← разные identity
```

МОГУТ ДОСПРОСИТЬ

- 01 Почему inline { } в JSX ломает React.memo?
- 02 Как сохранить identity вложенных объектов при иммутабельных обновлениях?
- 03 Что такое structural sharing и при чём тут Immer?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/object-copy>

ДЕНЬ 6

НАГРУЗКА

СРЕДНИЙ

3 ВОПРОСА

КОПИРОВАНИЕ**SHALLOW vs DEEP****structuredClone****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Shallow vs deep, structured cloning, и почему `JSON.parse(JSON.stringify)` – не панацея.

ВОПРОСЫ ДНЯ

Q12 Чем shallow copy отличается от deep copy?

Q13 Что такое structured cloning?

Q14 Какие проблемы есть у `JSON.parse(JSON.stringify(obj))`?

Чем shallow copy отличается от deep copy?

ОПИСАНИЕ

Shallow copy – копия только верхнего уровня объекта/массива: новые контейнер с теми же ссылками на вложенные объекты. Поменяешь вложенное – изменение видно и в оригинале. Deep copy – рекурсивная копия со всеми уровнями. Все вложенные объекты тоже копируются. Изменения в копии не влияют на оригинал. Главные инструменты: `spread (...)` / `Object.assign` – shallow; `structuredClone`, `lodash.cloneDeep`, `JSON.parse(JSON.stringify)` – deep (со своими нюансами). Глубокое копирование дороже по времени и памяти, поэтому его делают только когда правда нужно.

КАК РАБОТАЕТ

- 01 `Spread {...obj}` и `Object.assign({}, obj)` – копируют только собственные перечислимые свойства верхнего уровня, без рекурсии. Прототип не копируется.
- 02 `Array.from(arr)` и `[...arr]` – shallow для массивов.
- 03 `structuredClone` – встроенный API, делает deep по алгоритму structured clone (см. Q13). Поддерживается в Node 17+ и современных браузерах.
- 04 `JSON.parse(JSON.stringify(obj))` – старый трюк для deep. Работает только для plain JSON-данных, теряет много типов (см. Q14).
- 05 `lodash.cloneDeep` – рекурсивный обход, поддерживает Date, Map, Set, RegExp, циклы. Дорогой по перфомансу.
- 06 Для immutable-обновлений библиотеки (Immer) делают structural sharing: копируют только изменённую ветку, остальное оставляют по ссылке.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Иммутабельные обновления state в Redux/Zustand: `spread` верхнего уровня + точечное копирование изменённой части.
- 02 React: `setState({ ...state, x: newX })` – shallow для обновления одного поля.
- 03 Сериализация для отправки по сети, сохранения в localStorage – JSON-вариант почти всегда.
- 04 Тесты – deep clone тестовых фикстур, чтобы тесты не влияли друг на друга.

ПОДВОДНЫЕ КАМНИ

- 01 Самая частая ошибка – `spread+spread` думают что делает deep, а это всё shallow. Внутренние объекты остаются общими.
- 02 `JSON.parse(JSON.stringify)` теряет undefined, функции, Symbol, Date становится строкой, циклические ссылки выбрасывают ошибку.
- 03 `Object.assign` не копирует прототип и геттеры/сеттеры так, как ожидается. Для классов это сюрприз.
- 04 `structuredClone` не работает с функциями, классами, prototype – только с данными.

05 `lodash.cloneDeep` – медленный для больших объектов. Если проблема – сменить структуру или использовать `Immer`.

КОД

// Spread – shallow

```
const a = { x: 1, nested: { y: 2 } };
const b = { ...a };
b.x = 9;           // a.x = 1 (ок, верх скопирован)
b.nested.y = 9;    // a.nested.y = 9 (общая ссылка!)
```

// `structuredClone` – deep, встроенный

```
const a = { date: new Date(), set: new Set([1,2]) };
const b = structuredClone(a);
b.date instanceof Date; // true
b.set instanceof Set;   // true
```

// JSON-deep – кривой и быстрый

```
const a = { x: 1, fn: () => 1, when: new Date() };
const b = JSON.parse(JSON.stringify(a));
b.fn;           // undefined – функция потеряна
b.when;         // строка, не Date
```

// `lodash.cloneDeep`

```
import cloneDeep from 'lodash/cloneDeep';
const b = cloneDeep(a); // тот же тип у date, set, regexp
```

МОГУТ ДОСПРОСИТЬ

- 01 Что копирует `Object.assign` – прототип, геттеры/сеттеры?
- 02 Какие типы умеет `structuredClone`?
- 03 Когда лучше `Immer`, а когда – ручной `clone`?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/object-copy>

Что такое structured cloning?

ОПИСАНИЕ

Structured Cloning Algorithm – это алгоритм глубокого копирования, описанный в HTML-спецификации. Изначально его придумали для передачи данных между потоками (postMessage, Web Workers, Service Worker), и для записи в IndexedDB. С 2022 года он доступен напрямую в виде функции structuredClone(value). Поддерживает Date, RegExp, Map, Set, ArrayBuffer, Blob, File, циклические ссылки. Не поддерживает функции, классы, прототипы и DOM-узлы – при попытке клонировать такие данные кидает DataCloneError.

КАК РАБОТАЕТ

- 01 structuredClone(value, options?) – глобальная функция в браузере и Node 17+. Возвращает глубокую копию.
- 02 Сохраняет тип: Date остаётся Date, Map – Map, Set – Set. JSON-вариант это теряет.
- 03 Корректно обрабатывает циклические ссылки: a.self = a → клон тоже будет b.self === b.
- 04 options.transfer – список Transferable объектов (ArrayBuffer, MessagePort, ImageBitmap), которые ПЕРЕДАЮТСЯ, не копируются. После transfer оригинал становится detached.
- 05 Не поддерживает функции, классы (теряет prototype), DOM-узлы, Error (раньше не клонировался, в новых ревизиях – да). При несовместимом значении – DataCloneError.
- 06 Работает чуть медленнее, чем JSON.parse/JSON.stringify для простых данных, но сильно быстрее ручных рекурсивных обходов с поддержкой типов.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Глубокое копирование plain JS-данных без потери типов – первый выбор в современном коде.
- 02 postMessage между Worker и main thread – там structured clone применяется автоматически.
- 03 IndexedDB – данные в неё пишутся через тот же алгоритм.
- 04 Тесты, где нужны независимые фикстуры с Map/Set/Date.

ПОДВОДНЫЕ КАМНИ

- 01 Функции и методы класса не копируются. После structuredClone от instance-а класса вы получите plain-объект без методов.
- 02 DOM-узлы не копируются – DataCloneError.
- 03 Transferable: после options.transfer оригинал нельзя использовать. Это by design, но удивляет.
- 04 В Node 16 нет structuredClone – нужна полифилл-обёртка или встроенная util.types.

05 Большие объекты копируются полностью – для горячего пути это может быть дорого. Альтернатива – immer / structural sharing.

КОД

// Базовое использование

```
const a = { date: new Date(), set: new Set([1,2,3]) };
const b = structuredClone(a);
b.set.add(4);
a.set.has(4); // false
```

// Циклические ссылки – ок

```
const a = { x: 1 };
a.self = a;
const b = structuredClone(a);
b.self === b; // true
```

// Функции – DataCloneError

```
structuredClone({ fn: () => 1 });
// DataCloneError
```

// Transfer – передача без копирования

```
const buf = new ArrayBuffer(1024);
const clone = structuredClone(buf, { transfer: [buf] });
// buf теперь detached, использование = ошибка
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем structuredClone отличается от.postMessage?
- 02 Что значит transferable и какие объекты transferable?
- 03 Что произойдёт с instance класса после structuredClone?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/object-copy>

Какие проблемы есть у `JSON.parse(JSON.stringify(obj))`?

ОПИСАНИЕ

Это популярный, простой и быстрый способ сделать `deep clone`, но у него много ограничений – он работает только для `plain JSON`-совместимых данных. Любые другие типы либо теряются, либо превращаются в строки, либо вообще выбрасывают ошибку. Поэтому в современном коде его всё чаще заменяют на `structuredClone` (см. Q13). Главные проблемы: `undefined` и функции теряются; `Symbol` теряется; `Date` становится строкой; `Map`, `Set`, `RegExp` превращаются в пустые объекты или строки; циклические ссылки → `TypeError`; `BigInt` → `TypeError`; ключи-`Symbol` теряются.

КАК РАБОТАЕТ

- 01 `JSON.stringify` обходит объект и выбрасывает в строку только те значения, которые поддерживает: `string`, `number`, `boolean`, `null`, `plain-object`, `array`.
- 02 `undefined` и функции в значениях ОБЪЕКТА выбрасываются (поле пропадает). Внутри МАССИВА – становятся `null`.
- 03 `Date` вызывает `.toISOString()` – превращается в строку. После `JSON.parse` это уже не `Date`, а строка.
- 04 `RegExp` превращается в пустой объект `{}`.
- 05 `Map` и `Set` – тоже пустой `{}` (у них нет `.toJSON`).
- 06 `BigInt` – `TypeError`, `JSON` не умеет сериализовать.
- 07 Циклические ссылки – `TypeError: Converting circular structure to JSON`.
- 08 Ключи-`Symbol` игнорируются.
- 09 Прототипы и геттеры/сеттеры не учитываются – только собственные перечислимые свойства.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Для простого `deep clone plain-data` в скриптах, где гарантировано нет дат, мап, сетов и прочего.
- 02 Когда `structuredClone` недоступен (старый Node, IE), и не хочется тащить `lodash.cloneDeep`.
- 03 В тестах, для отделения тестовой фикстуры от исходной – если данные простые.
- 04 Иногда для нормализации (через `JSON.stringify` со `replacer`) – но это редкий случай.

ПОДВОДНЫЕ КАМНИ

- 01 `Date` становится строкой – частая причина «оно работало в тестах, упало в проде».
- 02 `Map` и `Set` теряются – критично для современных `Redux/Zustand`-стейтов с этими типами.

- 03 Циклические ссылки → исключение. Если `JSON.stringify` внезапно падает — ищем циклы.
- 04 `BigInt` → `TypeError`. Современные API часто возвращают `bigint`, и стандартный JSON их не глотает.
- 05 Поле `undefined` пропадает — потенциальная потеря «опционально не задано», иногда отличающегося от отсутствия поля.
- 06 Это однопоточный синхронный обход, на больших структурах может заметно тормозить.

КОД

// Что теряется

```
const obj = {
  a: undefined,
  fn: () => 1,
  when: new Date(),
  set: new Set([1,2]),
  big: 10n,
};
JSON.stringify(obj);
// SyntaxError: Do not know how to serialize a BigInt
// если убрать big — undefined и fn пропадут,
// when станет строкой, set — пустой объект
```

// Циклы — `TypeError`

```
const a = {}; a.self = a;
JSON.stringify(a);
// TypeError: Converting circular structure to JSON
```

// Когда это безопасно

```
// только plain-данные без дат и мап
const safe = { x: 1, y: [1,2,{z:3}] };
const clone = JSON.parse(JSON.stringify(safe));
```

// Современная замена

```
const clone = structuredClone(value);
```

МОГУТ ДОСПРОСИТЬ

- 01 Что происходит с `undefined` в массиве и в объекте при `JSON.stringify`?
- 02 Как обойти ограничение по циклам — `replacer` или `WeakSet`?
- 03 Когда осознанно выбрать `JSON-deep`, а когда `structuredClone`?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/object-copy>

ИММУТАБЕЛЬНОСТЬ**REACT****Map / WeakMap****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Закрепление: почему мутация ломает React-рендер, зачем immutability, чем Map отличается от объекта, когда WeakMap.

ВОПРОСЫ ДНЯ

- Q17 Почему мутация объекта может сломать React-rendering?
- Q18 Что такое immutability и зачем она нужна?
- Q19 Как работает WeakMap?
- Q20 Когда использовать Map, а когда обычный object?

ВОПРОС Q17

Почему мутация объекта может сломать React-rendering?

ОПИСАНИЕ

React определяет, что компонент нужно перерисовать, сравнивая ссылки на state и props между рендерами. Если ты мутируешь объект (push в массив, изменение поля), ссылка остаётся той же — для React ничего не изменилось. В итоге setState с тем же объектом, useState/useReducer, useМемо/useCallback с deps — все они «не видят» изменений. Возможны ровно противоположные баги: мутация во вне, через общую ссылку, проникает в другие компоненты, и предсказуемость ломается. Поэтому в React всегда работают с иммутабельными обновлениями.

КАК РАБОТАЕТ

- 01 useState/useReducer хранят ссылку. setState(newValue) вызывает рендер только если `Object.is(prev, newValue) === false`.
- 02 useМемо/useCallback пересчитываются только при смене ссылки одной из deps.
- 03 React.мемо по умолчанию делает shallow compare props — если ссылки prop не изменились, рендер пропускается.
- 04 Поэтому мутация объекта (`state.x = 1`) с последующим `setState(state)` — рендер НЕ происходит: тот же объект.
- 05 Иммутабельные обновления: spread верхнего уровня, новый объект для каждой изменённой ветки. Ссылка меняется → `Object.is` видит разницу → рендер.
- 06 Библиотеки уровня Immer позволяют писать «мутации» внутри draft, на выходе получают новый объект через structural sharing.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любое обновление state в Redux/Zustand/useReducer — иммутабельно, без исключений.
- 02 Передача объекта в мемо-обёрнутый компонент — стабилизировать ссылку через useМемо.
- 03 Работа с массивами в state: вместо push/pop/splice — `[...arr, x]`, `arr.filter(...)`, `arr.map(...)`.
- 04 Кэши/мемоизация в selectors — иммутабельные обновления + ссылочное равенство = быстрая проверка «не изменилось ли».

ПОДВОДНЫЕ КАМНИ

- 01 Прямой push/pop/splice мутирует — ссылка остаётся та же.
- 02 `Object.assign(state, {x:1})` — мутация! Нужно `Object.assign({}, state, {x:1})` или `{...state, x:1}`.
- 03 Spread не deep — при глубоком обновлении нужно копировать цепочку до изменённого узла, иначе вложенные объекты общие со старым state.
- 04 Мутирование state снаружи компонента (через общую ссылку, глобальный кэш) — самый коварный случай. State обновлён, но React не знает.

- 05 В классах `this.state.x = 1` – анти-паттерн, не вызывает рендер. Только `setState`.
- 06 TypeScript Readonly помогает поймать мутации на этапе компиляции – полезный приём.
-

код

// Мутация – рендер не вызывается

```
const [user, setUser] = useState({ name: 'A', age: 1 });
// плохо – мутация
user.age = 2;
setUser(user); // та же ссылка → React пропустит
```

// Иммуutableное обновление

```
setUser({ ...user, age: 2 });
// новый объект, ссылка другая → React рендерит
```

// Глубокое обновление

```
setForm({
  ...form,
  address: { ...form.address, city: 'New' },
});
```

// Immer – пишем «мутации», получаем иммутабельный результат

```
import produce from 'immer';
setForm(produce(form, draft => {
  draft.address.city = 'New';
}));
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое structural sharing и при чём тут Immer?
- 02 Почему React.memo не работает с inline {} в JSX?
- 03 Как TypeScript помогает не мутировать state?

ВОПРОС Q18

Что такое immutability и зачем она нужна?

ОПИСАНИЕ

Immutability (неизменяемость) – практика, при которой данные нельзя изменить после создания. Любое «обновление» создаёт новый объект, а старый остаётся как был. Это даёт сразу три вещи: предсказуемость (никто не подменит данные за твоей спиной), простую проверку «изменилось ли» через ссылочное равенство, и удобную работу с историей (undo/redo, time-travel debugging). Главные минусы – больше памяти на промежуточные копии и чуть больше кода на обновления. В современных JS-фреймворках (React, Redux, реактивные stores) immutability – стандарт де-факто.

КАК РАБОТАЕТ

- 01 Имутабельные операции возвращают НОВЫЙ объект, не меняя исходный: `arr.concat(x)`, `arr.filter(p)`, `arr.map(f)`, `{ ...obj, key: value }`.
- 02 Мутлирующие операции (изменяют исходное): `push`, `pop`, `splice`, `shift`, `sort`, `reverse`, `Object.assign(target, source)`, `delete obj.x`.
- 03 В JS нет строго иммутабельных типов кроме примитивов. `Object.freeze` делает объект «замороженным» (`writable: false`), но это shallow – вложенные остаются мутабельными.
- 04 Структурное разделение (structural sharing): после иммутабельного обновления неизменённые ветки остаются теми же по ссылке. Это экономит память.
- 05 Библиотеки: Immer (draft → producer), Immutable.js (свои Record/Map/List), Mori (более старый аналог).
- 06 TypeScript: `readonly modifier`, `ReadonlyArray<T>`, `Readonly<T>` – статическая защита от мутаций.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Redux, Zustand, MobX (selectively), useReducer – везде иммутабельный апдейт.
- 02 Селекторы и мемоизация – ссылочное равенство как дешёвая проверка изменения.
- 03 Time-travel debugging (Redux DevTools) – благодаря иммутабельности легко прокручивать историю.
- 04 Concurrent rendering в React 18 – для безопасности иммутабельный state обязателен.
- 05 Шардированные state в SSR / server actions – для корректной сериализации между клиентом и сервером.

ПОДВОДНЫЕ КАМНИ

- 01 `Object.freeze` – shallow, не глубокий. Замораживает только верхний уровень.
- 02 Глубокая иммутабельность вручную – много spread-кода на обновления вложенных полей. Лекарство – Immer.
- 03 Иммутабельность ≠ дешевизна копирования. Для огромных списков нужны структурно-разделяющие коллекции.

- 04 Иногда для UI перфоманса разрешена локальная мутация перед `setState` – пишем в новый клон, потом `setState`. Это ок, потому что снаружи объект иммутабелен.
- 05 Иммутабельность не решает `race condition` или `async-баги` – это другая ось.
- 06 Спор «`immutability vs perf`» решается в пользу `immutability` по умолчанию, `perf` – оптимизировать точно.

код

// Иммутабельный push

```
const arr = [1, 2, 3];
const next = [...arr, 4];
// arr остался [1,2,3], next – [1,2,3,4]
```

// `Object.freeze` – shallow

```
const o = Object.freeze({ x: 1, n: { y: 2 } });
o.x = 9;           // молча игнор (или TypeError в strict)
o.n.y = 9;         // прошло – n не заморожен
```

// Глубокая заморозка

```
function deepFreeze(o) {
  for (const k of Object.keys(o)) {
    const v = o[k];
    if (typeof v === 'object' && v !== null) deepFreeze(v);
  }
  return Object.freeze(o);
}
```

// TypeScript readonly

```
type User = { readonly id: string; name: string };
// u.id = '...' – ошибка компиляции
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `Object.freeze` отличается от `Object.seal` и `Object.preventExtensions`?
- 02 Что такое `structural sharing` на практике?
- 03 Когда использовать `Immutable.js`, а когда `Immer`?

ВОПРОС Q19

Как работает WeakMap?

ОПИСАНИЕ

WeakMap — это коллекция типа Map, но с одним важным отличием: ключами могут быть только объекты, и WeakMap хранит «слабые» ссылки на них. Это значит, что если ключ больше нигде в коде не используется, GC может его удалить, и запись в WeakMap пропадёт автоматически. У WeakMap нет методов перебора (нет `.keys()`, `.values()`, `.entries()`, нет `.size`), потому что состав может меняться в любой момент из-за GC. Это делает его идеальным для метаданных, кэшей и приватных полей, привязанных к объектам, без рисков удержания памяти.

КАК РАБОТАЕТ

- 01 Ключи — только объекты (или Symbol с registry, в новых версиях стандарта). Примитив-ключ — TypeError.
- 02 Значения — что угодно: примитивы, объекты, функции.
- 03 Если на объект-ключ нет других ссылок в программе, GC может его собрать. Запись в WeakMap для этого ключа пропадает «как бы сама».
- 04 API минимальный: get, set, has, delete. НЕТ keys/values/entries/size/iterator.
- 05 Не итерируется — потому что состав может меняться несинхронно с кодом.
- 06 Похожий тип — WeakSet (только ключи без значений), и WeakRef (одна слабая ссылка на конкретный объект).
- 07 Не подходит для случаев, где нужно перебрать все записи или знать размер.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Метаданные для DOM-узлов или объектов, не модифицируя их: node → meta. Когда DOM-узел удалится, meta освободится.
- 02 Кэши результатов вычислений по объекту-аргументу: weakMap.get(obj) ?? (weakMap.set(obj, calc(obj)), ...).
- 03 Реализация приватных полей в классах (до приватных полей ES2022) через замыкания + WeakMap.
- 04 Связь instance ↔ метаданные в реактивных библиотеках (Vue 3 reactive, Solid, Mobx) — все используют WeakMap.

ПОДВОДНЫЕ КАМНИ

- 01 Нет .size и нет итерации — иногда приходится держать вторую обычную Map для перебора, что уже само по себе удерживает.
- 02 Ключ — только объект. Чтобы привязать к строке/числу — нужно завернуть в объект-обёртку (что обычно костыль).
- 03 GC сработает «когда-то». Удаление записи не мгновенное, и точное время не гарантировано.

04 WeakMap нельзя сериализовать в JSON.

05 Не путать с обычной Map: Map удерживает ключи, WeakMap — нет.

КОД

// Кэш по объекту

```
const cache = new WeakMap();
function getMeta(obj) {
  if (!cache.has(obj)) cache.set(obj, expensive(obj));
  return cache.get(obj);
}
// если obj больше нигде не нужен — кэш пропадёт сам
```

// Приватные поля до ES2022

```
const _password = new WeakMap();
class User {
  constructor(pwd) { _password.set(this, pwd); }
  check(p) { return _password.get(this) === p; }
}
```

// Метаданные DOM-узлов без утечек

```
const meta = new WeakMap();
function attach(node, info) { meta.set(node, info); }
// узел удалён из DOM, ссылок больше нет → GC очистит
```

МОГУТ ДОСПРОСИТЬ

01 Чем WeakMap отличается от WeakSet?

02 Когда нужен WeakRef вместо WeakMap?

03 Можно ли использовать примитив как ключ в WeakMap?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/weakmap-weakset>

ВОПРОС Q20

Когда использовать Map, а когда обычный object?

ОПИСАНИЕ

`object` – самая базовая структура `key-value` в JS, но у неё много нюансов: ключи всегда строки или `Symbol`; есть прототип, из которого случайно «пролезают» свойства; нет встроенного способа узнать число записей; нет гарантий порядка для числовых ключей. `Map` – отдельная коллекция: ключи могут быть любого типа (включая объекты), есть `.size`, итерация в порядке вставки, нет случайного наследования. Поэтому: `object` – когда нужен «структурный» объект с фиксированным набором полей; `Map` – когда коллекция произвольных пар, особенно с ключами не-строками или с частыми операциями вставки/удаления.

КАК РАБОТАЕТ

- 01 `object`: ключи – `string` или `Symbol`. Все остальные приводятся к строке (например, ключ `1` становится `'1'`, ключ объект – `'[object Object]'`).
- 02 `object` имеет прототип (`Object.prototype`), и его свойства тоже видны через `for-in` без `hasOwn`-проверки. `Object.create(null)` даёт «чистый» объект.
- 03 `Map`: ключи любого типа. `Identity`-сравнение для объектов.
- 04 `Map` поддерживает `.size` (без обхода). `object` – нужно `Object.keys(o).length`.
- 05 Итерация `Map` – по порядку вставки. У объекта порядок чуть хитрее: сначала числовые ключи в возрастающем порядке, потом строковые в порядке вставки, потом символьные.
- 06 Производительность: `Map` обычно быстрее для частых вставок/удалений, особенно с большим `N`. `object` часто оптимизируется V8 как `hidden class` – быстрее для стабильных полей.
- 07 `JSON.stringify` работает с `object`, не с `Map` (`Map` превращается в пустой объект). Поэтому для сериализации часто `object`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 `object` – структурный тип: `Person`, `Config`, `Props`. Фиксированный набор полей.
- 02 `object` – когда нужен `JSON-friendly` результат (`API` ответы, `localStorage`).
- 03 `Map` – динамический словарь произвольных пар, особенно с не-строковыми ключами (объекты, числа без приведения к строке).
- 04 `Map` – когда часто `.size`, `.has` с гарантиями, без рисков конфликта с прототипом.
- 05 `Map` – кэши и реестры внутри библиотеки, где не нужна сериализация.
- 06 `Set` – когда нужно «множество» (только ключи), часто `.has`.

ПОДВОДНЫЕ КАМНИ

- 01 `Object.create(null)` – нужен, если используешь объект как словарь и боишься конфликта с прототипными свойствами (`toString`, `constructor`).

- 02 for-in перечисляет наследуемые свойства тоже – нужно `hasOwn`. `Object.keys` / `.entries` / `.values` – только собственные.
- 03 `object.x = 1` при `x = '__proto__' / 'constructor'` – может быть гадость в зависимости от движка. Для словарей лучше `Map` или `null-prototype`.
- 04 `Map` не сериализуется `JSON.stringify` напрямую – нужно `[...map.entries()]` или собственный `replacer`.
- 05 Если ключи всегда строки и количество маленькое – `object` обычно быстрее.
- 06 Нельзя смешивать паттерны: «object как структура» и «object как словарь» – выбирай один сценарий.

код

// Объект как структура

```
const user = { id: 1, name: 'Alice', age: 30 };
```

// Map как словарь с произвольным ключом

```
const meta = new Map();
meta.set(domNode, { highlighted: true });
meta.set({ x: 1 }, 'inline-config');
```

// Грабли с прототипом

```
const dict = {};
dict.toString; // function (наследовано)
'toString' in dict; // true ([])
Object.hasOwn(dict, 'toString'); // false
```

// null-prototype для безопасного словаря

```
const dict = Object.create(null);
dict.toString = 'safe'; // никаких конфликтов
```

// Конвертация между Map и object

```
const o = Object.fromEntries(map);
const m = new Map(Object.entries(obj));
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое `hidden class` в V8 и при чём тут `object`?
- 02 Почему `Object.create(null)` полезен для словарей?
- 03 Чем `Map` отличается от `Set` по интерфейсу?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/map-set>

ФИНАЛЬНЫЙ ЧЕК - ЛИСТ

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ Своими словами объяснить, что такое замыкание (Q1)

- ☐ Описать lexical scope и scope chain (Q6)

- ☐ Сравнить var, let, const по scope, hoisting и TDZ (Q7, Q8)

- ☐ Объяснить stale closure и три способа его лечения в React (Q3, Q4, Q5)

- ☐ Описать, как замыкания удерживают память и где это становится утечкой (Q9, Q10, Q11)

- ☐ Сравнить shallow и deep copy, описать ограничения JSON-варианта (Q12, Q14)

- ☐ Знать, что такое structuredClone и какие типы он умеет (Q13)

- ☐ Сравнить ссылочное и структурное равенство, объяснить React.memo (Q15, Q16)

- ☐ Объяснить, почему мутация ломает React-рендер, и зачем immutability (Q17, Q18)

- ☐ Описать WeakMap: разница с Map, когда выбирать (Q19, Q20)

EVENT LOOP / ASYNC МЕТОДИЧКА v1

МАТЕРИАЛ ОПИРАЕТСЯ НА learn.javascript.ru